



Esercitazione 3

Basi ortonormali: Base standard e DCT (Parte seconda)



Introduzione

L'esercitazione introduce il concetto di **basi ortonormali** applicate alla rappresentazione dei segnali e delle immagini digitali, con particolare attenzione alla **Trasformata Discreta del Coseno (DCT)**.

L'obiettivo è quello di verificare sperimentalmente alcune proprietà fondamentali delle basi e della matrice di trasformazione DCT, confrontarle con la base standard e studiarne l'applicazione pratica nell'elaborazione delle immagini.

Inoltre, si affronta il tema della **compressione delle immagini** tramite approssimazioni sui coefficienti della DCT, mettendo in evidenza l'importanza delle basi ortogonali per ottenere rappresentazioni compatte ed efficienti.

L'obiettivo finale è acquisire strumenti teorici e computazionali per comprendere come le basi ortonormali e la DCT possano essere utilizzate nell'analisi, nella ricostruzione e nella compressione dei segnali digitali.



Riassunto Esercitazione 3

In questa esercitazione si è lavorato sull'immagine `cameraman.jpg` (verificata come scala di grigi 64×64 , valori interi in $[0, 255]$) per confrontare la base canonica e la base DCT bidimensionale e per studiare l'effetto di diverse strategie di approssimazione sui coefficienti.

Gli step principali svolti sono stati:

- Lettura e controllo dell'immagine (`cameraman.jpg`): conversione a scala di grigi quando necessario e verifica delle dimensioni e del range di valori (64×64 , valori in $[0, 255]$).
- Costruzione della **base canonica** (atomi impulso, una immagine con un solo pixel = 1) e loro visualizzazione come elementi della base.
- Costruzione della **matrice DCT 1-D** di dimensione $N \times N$ (con $N = 64$) e generazione della **base DCT 2-D** tramite prodotto tensore (outer product) delle direzioni 1-D; visualizzazione di alcuni atomi DCT 1-D e 2-D per evidenziare le differenze strutturali rispetto alla base canonica.
- Verifica numerica dell'ortogonalità/ortonormalità della DCT: controllo che $C^T C \approx I$ (verifica tramite norma della differenza).
- Calcolo dei **coefficienti** dell'immagine nella base DCT (trasformata DCT 2-D) e ricostruzione mediante la trasformata inversa; verifica della ricostruzione esatta nel caso di conservazione di tutti i coefficienti.
- Esperimenti di **compressione / approssimazione**:
 - **Approssimazione lineare**: conservare un prefissato insieme di coefficienti (es. primi k secondo un'ordinamento di frequenza) e ricostruire l'immagine; osservazione dell'effetto visivo e del decadimento dell'errore al variare di k .
 - **Approssimazione non lineare (best-k)**: selezione dei k coefficienti di maggior modulo (sparse approximation) e ricostruzione con quei coefficienti; calcolo e confronto dell'errore relativo.
- Confronto tra base canonica e base DCT (sia qualitativo — immagini ricostruite — sia quantitativo — errore relativo). In particolare si osserva che la DCT concentra l'energia in pochi coefficienti e quindi, per lo stesso numero di coefficienti conservati, l'approssimazione DCT (soprattutto non lineare) dà tipicamente errori molto più piccoli rispetto alla base canonica.

Obiettivo raggiunto: dimostrare sperimentalmente che la rappresentazione DCT è più compatta per immagini natural-like (come il cameraman): concentra energia nei coefficienti a bassa frequenza e rende più efficace la compressione tramite selezione dei coefficienti significativi.

```
In [1]: import numpy as np
import matplotlib.pyplot as plt
import scipy.io as sio
```



Esercizio 1

In questo esercizio consideriamo due tipi di base per la rappresentazione di immagini: la base canonica e la base DCT bidimensionale.

(a) Leggere l'immagine `cameraman.jpg` usando il comando

```
np.array(plt.imread('cameraman.jpg'), dtype='float64')
utilizzando le librerie numpy e matplotlib.
```

(b) Visualizzare l'immagine usando il comando `imshow` della libreria `matplotlib`.

(c) Verificare che l'immagine caricata è una immagine in scala di grigi di dimensione 64×64 pixel. (Controllare che ciascun pixel abbia un valore intero compreso tra 0 e 255.)

(d) La base canonica di questo spazio è formata dall'insieme di matrici dove un solo elemento è uguale a uno e tutti gli altri sono zero.

Supponiamo di voler trasmettere questa immagine. L'immagine viene rappresentata utilizzando la base canonica e vengono trasmessi i coefficienti $c_i = \langle I, e_i \rangle$, con $1 \leq i \leq 4096$.

Supponiamo che ci sia un errore nella trasmissione e solo una parte dei coefficienti venga trasmessa. L'immagine ricevuta è un'approssimazione dell'originale con i coefficienti mancanti posti a zero.

Visualizzare l'immagine ricevuta e calcolare l'errore relativo di trasmissione.

(e) Considerare ora una base diversa da quella canonica. Leggere il file `DCT.mat` usando

```
sio.loadmat('DCT.mat')['B']
```

dalla libreria `scipy.io`. Il file contiene una matrice tridimensionale `B` contenente la base della DCT.

Per ottenere il i -esimo elemento della base usare:

```
bi = np.reshape(B[i, :, :], [64, 64])
```

Visualizzare con `imshow` gli elementi 0, 1, 2, 3, 64, 65, 66, 67.

(f) Calcolare l'errore di approssimazione prendendo solo la **prima metà dei coefficienti** dell'immagine rispetto alla **base canonica**.

Ripetere il calcolo con la **base DCT**, usando

$$I_{rec} = \sum_{i=1}^{4096} c_i b_i$$

e confrontare gli errori. Quale base fornisce una migliore approssimazione?

(g) Visualizzare in ordine decrescente il **valore assoluto** dei primi 2048 coefficienti dell'immagine rispetto alla base DCT.

Approssimare l'immagine usando i primi $k = 2048$ coefficienti **più significativi**, visualizzare l'approssimazione e calcolare l'errore relativo.

```
In [2]: # -----
# (a) Lettura immagine
# -----
I = np.array(plt.imread('cameraman.jpg'), dtype='float64')
print(f"✅ Immagine letta: shape = {I.shape}, dtype = {I.dtype}")

# Se RGB, prendiamo il primo canale
if I.ndim == 3:
    I = I[:, :, 0]
    print("📺 Immagine RGB: selezionato il canale 0 per scala di grigi")

# -----
# (b) Visualizzazione
# -----
plt.figure(figsize=(4, 4))
plt.imshow(I, cmap='gray', vmin=0, vmax=255)
plt.title("Immagine originale (cameraman)")
plt.axis('off')
plt.show()

# -----
# (c) Verifica scala di grigi 64x64 e valori interi [0,255]
# -----
h, w = I.shape
is_int = np.all(np.floor(I) == I)
in_range = np.all((I >= 0) & (I <= 255))

print(f"📏 Dimensioni 64x64? {'OK' if (h, w)==(64,64) else 'NO'}")
print(f"🔢 Valori interi? {'OK' if is_int else 'NO'}")
print(f"🔢 Valori in [0,255]? {'OK' if in_range else 'NO'}")

# -----
# (d) Base canonica: trasmissione con cancellazione coefficiente
# -----
# Costruiamo i coefficienti ci = <I, ei>, ma in pratica
# se EI è la base canonica, ci = I.flatten()[i]
coeff_can = I.flatten().copy()
num = coeff_can.size
num_erasures = num // 4 # es. primo 25% trasmessi
idx = np.random.permutation(num)
to_zero = idx[num_erasures:]
coeff_rec_can = coeff_can.copy()
coeff_rec_can[to_zero] = 0

# Ricostruisco l'immagine
I_rec_can = coeff_rec_can.reshape(I.shape)
plt.figure(figsize=(4, 4))
plt.imshow(I_rec_can, cmap='gray', vmin=0, vmax=255)
plt.title("Ricevuta - base canonica")
plt.axis('off')
plt.show()

# Errore relativo
err_can = np.linalg.norm(I - I_rec_can) / np.linalg.norm(I)
print(f"❌ Errore relativo (base canonica): {err_can:.2e}")

# -----
# (e) Base DCT 2D: caricamento e visualizzazione elementi
# -----
B = sio.loadmat('DCT.mat')['B'] # B.shape = (4096,64,64)
indices = [0, 1, 2, 3, 64, 65, 66, 67]
plt.figure(figsize=(8, 4))
for i, idx in enumerate(indices):
    bi = B[idx, :, :].reshape(64, 64)
    ax = plt.subplot(2, 4, i+1)
    ax.imshow(bi, cmap='gray')
    ax.set_title(f"b[{idx}]")
    ax.axis('off')
plt.suptitle("Elementi selezionati base DCT 2D")
plt.tight_layout()
plt.show()

# -----
```

```

# (f) Approssimazione con prima metà dei coefficienti
# -----
# Calcolo tutti i coefficienti ci = <I, bi>
coef_dct = np.array([np.sum(I * B[i]) for i in range(B.shape[0])])

# Canonica: già fatto sopra come coeff_can
# DCT: tronco la seconda metà
coef_trunc = coef_dct.copy()
coef_trunc[len(coef_trunc)//2:] = 0

# Ricostruzione: I_rec_dct = sum ci * bi
I_rec_dct = np.tensordot(coef_trunc, B, axes=(0,0)) # forma (64,64)
plt.figure(figsize=(4,4))
plt.imshow(I_rec_dct, cmap='gray', vmin=0, vmax=255)
plt.title("Ricevuta - base DCT (1ª metà coeff.)")
plt.axis('off')
plt.show()

err_dct = np.linalg.norm(I - I_rec_dct) / np.linalg.norm(I)
print(f"❌ Errore relativo (base DCT): {err_dct:.2e}")
print(f"👉 Base migliore? {'DCT' if err_dct < err_can else 'canonica'})

# -----
# (g) Coefficienti DCT ordinati e approssimazione migliori k
# -----
# Ordino per valore assoluto discendente
order = np.argsort(np.abs(coef_dct))[:-1]
k = 2048
best_indices = order[:k]

# Grafico dei primi 2048 coefficienti
plt.figure(figsize=(6, 3))
plt.plot(np.abs(coef_dct[best_indices]))
plt.title("Primi 2048 coefficienti DCT più significativi")
plt.xlabel("Rank")
plt.ylabel("|c_i|")
plt.show()

# Approssimazione con i migliori k
coef_bestk = np.zeros_like(coef_dct)
coef_bestk[best_indices] = coef_dct[best_indices]
I_rec_bestk = np.tensordot(coef_bestk, B, axes=(0,0))

plt.figure(figsize=(4,4))
plt.imshow(I_rec_bestk, cmap='gray', vmin=0, vmax=255)
plt.title(f"Ricostruzione migliori {k} coeff.")
plt.axis('off')
plt.show()

err_bestk = np.linalg.norm(I - I_rec_bestk) / np.linalg.norm(I)
print(f"❌ Errore relativo (migliori {k} coeff.): {err_bestk:.2e}")

```

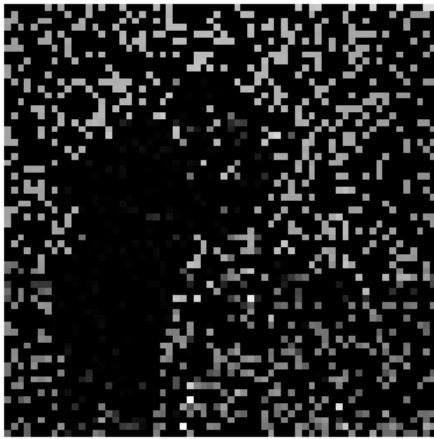
✅ Immagine letta: shape = (64, 64), dtype = float64

Immagine originale (cameraman)



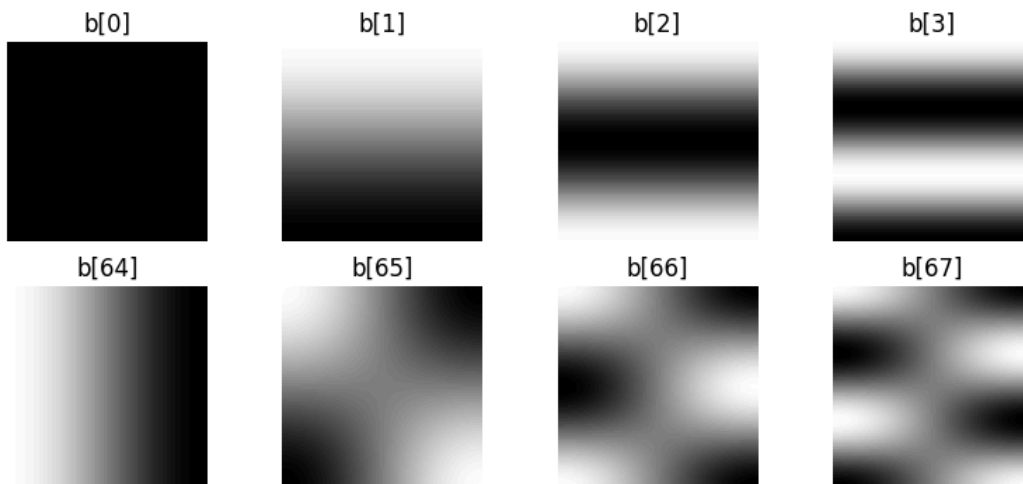
- 🔧 Dimensioni 64x64? OK
- 🔧 Valori interi? OK
- 🔧 Valori in [0,255]? OK

Ricevuta - base canonica



✗ Errore relativo (base canonica): $8.59e-01$

Elementi selezionati base DCT 2D



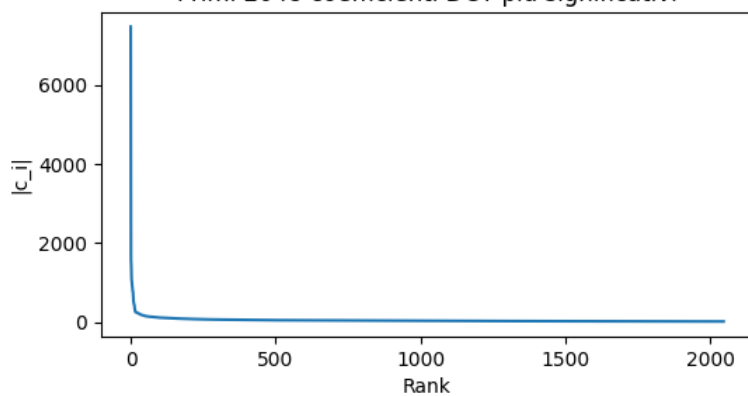
Ricevuta - base DCT (1ª metà coeff.)



✗ Errore relativo (base DCT): $1.38e-01$

👉 Base migliore? DCT

Primi 2048 coefficienti DCT più significativi



Ricostruzione migliori 2048 coeff.



✗ Errore relativo (migliori 2048 coeff.): $5.52e-02$

COMMENTO.

Nell'esercizio si è applicata la **compressione delle immagini tramite DCT**, analizzando sia il caso di approssimazione lineare (conservando un numero fisso di coefficienti) sia quello di approssimazione non lineare (conservando i coefficienti di maggiore ampiezza).

Dai risultati numerici e dalle immagini ricostruite si osserva che:

- già con un numero ridotto di coefficienti, l'immagine mantiene gran parte dell'informazione visiva essenziale;
- l'approssimazione **non lineare** risulta generalmente più efficiente, in quanto seleziona i coefficienti più rilevanti e riduce l'errore relativo rispetto all'approssimazione lineare;
- aumentando il numero di coefficienti conservati, l'errore di ricostruzione diminuisce sensibilmente, fino a ottenere un'immagine praticamente indistinguibile dall'originale.

Take home message: la DCT consente di sfruttare la ridondanza spaziale dell'immagine concentrando l'informazione nei coefficienti a bassa frequenza. Questo dimostra perché la DCT è alla base di molti algoritmi di compressione (es. JPEG): permette una rappresentazione compatta, mantenendo la qualità visiva con pochi coefficienti significativi.